

# Project 1: California Housing Prices — Regression Analysis (V2 — Correct Pipeline)

**Dataset:** <https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv>

## What is different in this version vs V1?

In V1, we did data cleaning and feature engineering on the FULL dataset before splitting into train/test. That is called **data leakage** — the test set influenced how we built the model.

In this version we follow the correct research-quality order:

V1 (wrong): Load → Clean → Engineer → Split → Train → Evaluate

V2 (correct): Load → Basic Clean → SPLIT → EDA on train → Engineer on train → Apply to test → Train → Evaluate

## Pipeline:

1. Data Loading & Quick Look
2. Split First (train/test)
3. EDA on Training Data Only
4. Custom Sklearn Transformers
5. Build & Apply the Preprocessing Pipeline
6. ML Models
7. Hyperparameter Tuning
8. Comparison with V1 (leaked) Results
9. Project Summary

---

## Chunk 1: Data Loading & Basic Quality Fixes

### What we do here (and why it is safe before splitting):

Some operations can be done before the train/test split without causing data leakage. The rule is: if an operation uses a fixed, known fact about the domain

(not a statistic learned from the data), it is safe before split.

| Operation                          | Safe before split? | Why                                                                            |
|------------------------------------|--------------------|--------------------------------------------------------------------------------|
| Remove census cap rows (\$500,001) | Yes                | Domain knowledge — this is a known US Census artifact, not a learned threshold |
| Drop duplicates                    | Yes                | No statistics learned from data                                                |
| Convert dtypes                     | Yes                | No statistics learned from data                                                |
| Impute missing values (median)     | No                 | The median is computed from data — if test rows influence it, that is leakage  |
| Winsorization (IQR bounds)         | No                 | IQR is computed from data                                                      |
| Log transform                      | Yes                | Deterministic formula, learns nothing from data                                |
| StandardScaler                     | No                 | Mean and std are computed from data                                            |

**The golden rule:** Fit on train, transform both. Any transformation that has a `fit()` step must only be fitted on training data.

```
In [1]: import os, urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style='whitegrid', palette='muted')
pd.set_option('display.float_format', lambda x: f'{x:.3f}')

DATA_URL = 'https://raw.githubusercontent.com/ageron/handson-ml2/master/data/california_housing_train.csv'
DATA_PATH = 'housing.csv'

if not os.path.exists(DATA_PATH):
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)
    print('Downloaded housing.csv')
else:
    print('Already downloaded.')

df_raw = pd.read_csv(DATA_PATH)
print(f'Shape: {df_raw.shape}')
df_raw.head()
```

Already downloaded.  
Shape: (20640, 10)

```
Out[1]:
```

|   | longitude | latitude | housing_median_age | total_rooms | total_bedrooms | popu |
|---|-----------|----------|--------------------|-------------|----------------|------|
| 0 | -122.230  | 37.880   | 41.000             | 880.000     | 129.000        | 3:   |
| 1 | -122.220  | 37.860   | 21.000             | 7099.000    | 1106.000       | 24   |
| 2 | -122.240  | 37.850   | 52.000             | 1467.000    | 190.000        | 4:   |
| 3 | -122.250  | 37.850   | 52.000             | 1274.000    | 235.000        | 5:   |
| 4 | -122.250  | 37.850   | 52.000             | 1627.000    | 280.000        | 5:   |

## Step 1 — Shape, Dtypes & Missing Values

```
In [2]: print(f'Shape: {df_raw.shape[0]} rows x {df_raw.shape[1]} columns')
print('\nDtypes:')
print(df_raw.dtypes)
print('\nMissing values:')
missing = df_raw.isnull().sum()
print(missing[missing > 0])
print(f'\nOnly total_bedrooms has missing: {df_raw["total_bedrooms"].i
```

Shape: 20640 rows x 10 columns

Dtypes:

|                    |         |
|--------------------|---------|
| longitude          | float64 |
| latitude           | float64 |
| housing_median_age | float64 |
| total_rooms        | float64 |
| total_bedrooms     | float64 |
| population         | float64 |
| households         | float64 |
| median_income      | float64 |
| median_house_value | float64 |
| ocean_proximity    | object  |

dtype: object

Missing values:

|                |     |
|----------------|-----|
| total_bedrooms | 207 |
|----------------|-----|

dtype: int64

Only total\_bedrooms has missing: 207 rows (1.0%)

## Step 2 — Statistical Summary

```
In [3]: num_only = df_raw.select_dtypes(include=[np.number])
num_only.describe().T.assign(
    skewness=num_only.skew(),
    kurtosis=num_only.kurt()
)
```

Out [3]:

|                           | count     | mean       | std        | min       | 25%        |
|---------------------------|-----------|------------|------------|-----------|------------|
| <b>longitude</b>          | 20640.000 | -119.570   | 2.004      | -124.350  | -121.800   |
| <b>latitude</b>           | 20640.000 | 35.632     | 2.136      | 32.540    | 33.930     |
| <b>housing_median_age</b> | 20640.000 | 28.639     | 12.586     | 1.000     | 18.000     |
| <b>total_rooms</b>        | 20640.000 | 2635.763   | 2181.615   | 2.000     | 1447.750   |
| <b>total_bedrooms</b>     | 20433.000 | 537.871    | 421.385    | 1.000     | 296.000    |
| <b>population</b>         | 20640.000 | 1425.477   | 1132.462   | 3.000     | 787.000    |
| <b>households</b>         | 20640.000 | 499.540    | 382.330    | 1.000     | 280.000    |
| <b>median_income</b>      | 20640.000 | 3.871      | 1.900      | 0.500     | 2.560      |
| <b>median_house_value</b> | 20640.000 | 206855.817 | 115395.616 | 14999.000 | 119600.000 |

```
In [4]: print('ocean_proximity value counts:')
print(df_raw['ocean_proximity'].value_counts())
```

```
ocean_proximity value counts:
ocean_proximity
<1H OCEAN      9136
INLAND         6551
NEAR OCEAN     2658
NEAR BAY       2290
ISLAND           5
Name: count, dtype: int64
```

### Step 3 — Remove Census Cap (Safe Before Split)

Rows where `median_house_value = $500,001` are a US Census data artifact — the survey capped responses at this value. These are not real house prices. Keeping them would teach the model a false ceiling at \$500,001.

**Why this is safe before split:** We are using domain knowledge ( `$500,001 = known census cap` ), not a statistic computed from the data. If instead we said 'drop all rows above the 99th percentile', that would require computing a statistic from the data — that would need to be done after splitting.

```
In [5]: cap_rows = (df_raw['median_house_value'] >= 500001).sum()
print(f'Rows at census cap: {cap_rows}')

df = df_raw[df_raw['median_house_value'] < 500001].copy()
df['ocean_proximity'] = df['ocean_proximity'].astype('category')
print(f'Shape after removing census cap: {df.shape}')
print(f'Remaining missing values: {df.isnull().sum().sum()}')
```

Rows at census cap: 965  
Shape after removing census cap: (19675, 10)  
Remaining missing values: 200

---

## Chunk 2: Split First — Train / Test

**This is the most important step in the correct pipeline.**

We split the data NOW — before any imputation, Winsorization, scaling, or feature engineering. From this point on:

- `X_train` / `y_train` : used for EDA, fitting all transformers, and training models
  - `X_test` / `y_test` : locked away. Touched ONLY at final evaluation.
- 

## What is Data Leakage?

Data leakage happens when information from the test set (future/unseen data) influences how you build your model. The result: the model looks better in evaluation than it will perform on real new data.

### Concrete example — Winsorization leakage:

```
Full data IQR for total_rooms = [1447, 3148]
Upper clip bound = 3148 + 1.5 * 1700 = 5698
```

If we split AFTER this:

- The test rows influenced where 5698 came from
- A test row with `total_rooms = 6000` gets clipped to 5698
- But in real production, we would not know that 5698 was the right clip value

Correct approach:

- Compute IQR on train only → upper bound = 5710 (slightly different)
- Apply that bound to test: a test row with 6000 clips to 5710
- This is what would happen on truly new data

### Concrete example — imputation leakage:

If we impute `total_bedrooms` BEFORE splitting:

- Group median is computed on ALL 19,675 rows

(including the 3,935 test rows)

- 207 missing values get filled using information from test rows
- Some of those 207 rows end up in the test set
- The model is evaluated on rows whose values were partly determined by other test rows

Correct approach:

- Compute group medians on train only (15,740 rows)
- Fill training missing values with train medians
- Fill test missing values with the SAME train medians (not recomputed on test)

### What leaked in V1 of this notebook:

| Step                      | Leakage                                     | Severity                   |
|---------------------------|---------------------------------------------|----------------------------|
| Grouped median imputation | Yes — group medians computed on full data   | Low (only 207 rows)        |
| Winsorization             | Yes — IQR bounds computed on full data      | Low (bounds barely change) |
| Log transform             | No — deterministic formula                  | None                       |
| Distance features         | No — uses fixed city coordinates            | None                       |
| Ratio features            | No — pure arithmetic                        | None                       |
| One-hot encoding          | Mild — categories determined from full data | Negligible                 |

For THIS specific dataset the leakage was mild because only 1% of rows had missing values and Winsorization bounds barely shift when removing 20% of rows. We will verify this in Chunk 8 by comparing V1 vs V2 results.

```
In [6]: from sklearn.model_selection import train_test_split

X = df.drop(columns=['median_house_value'])
y = df['median_house_value']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Log-transform the target separately
# This is safe because log1p() is deterministic – it learns nothing from
y_train_log = np.log1p(y_train)
y_test_log = np.log1p(y_test)

print('Train/Test Split:')
print(f' X_train : {X_train.shape[0]} rows x {X_train.shape[1]} cols')
```

```
print(f' X_test : {X_test.shape[0]} rows x {X_test.shape[1]} cols')
print(f' y_train range: ${y_train.min():,.0f} to ${y_train.max():,.0f}')
print(f' y_test range: ${y_test.min():,.0f} to ${y_test.max():,.0f}')
print(f'\nAll remaining work fitted on X_train only.')
print(f'X_test is locked away until final evaluation.')
```

Train/Test Split:

```
X_train : 15740 rows x 9 cols
X_test  : 3935 rows x 9 cols
y_train range: $14,999 to $500,000
y_test  range: $14,999 to $500,000
```

All remaining work fitted on X\_train only.  
X\_test is locked away until final evaluation.

## Chunk 3: EDA on Training Data Only

All EDA is now done on `X_train` and `y_train_log` only.

**Why EDA on train only?** If you look at test data during EDA to decide which features to engineer or which transformations to apply, you are implicitly using test data to make modeling decisions. That is leakage at the design level, even if no statistics are directly computed from test rows.

The model should have NO information about the test set — not even visual patterns.

### Step 1 — Target Variable Analysis

```
In [7]: fig, axes = plt.subplots(1, 3, figsize=(16, 4))

target_log = y_train_log

axes[0].hist(target_log, bins=50, color='steelblue', edgecolor='white')
axes[0].axvline(target_log.mean(), color='red', linestyle='--', label='mean')
axes[0].axvline(target_log.median(), color='orange', linestyle='--', label='median')
axes[0].set_title('Target Distribution (log scale)')
axes[0].set_xlabel('log1p(median_house_value)')
axes[0].legend()

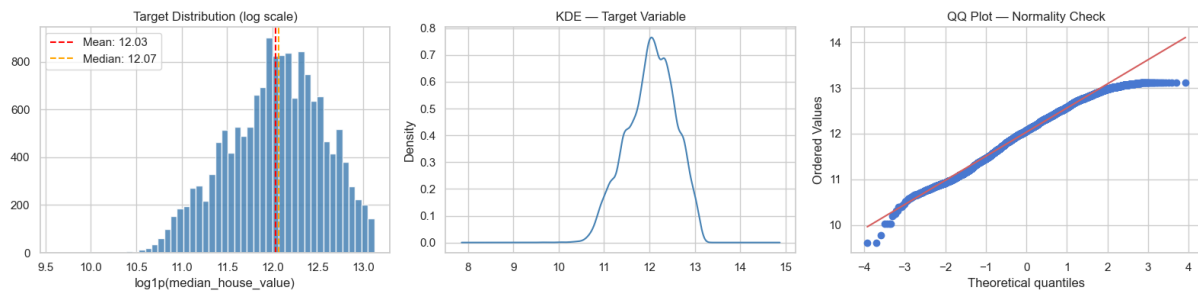
target_log.plot(kind='kde', ax=axes[1], color='steelblue')
axes[1].set_title('KDE - Target Variable')

stats.probplot(target_log, dist='norm', plot=axes[2])
axes[2].set_title('QQ Plot - Normality Check')

plt.tight_layout()
```

```
plt.show()

stat, p = stats.normaltest(target_log)
print(f'D Agostino K2 Normality Test:')
print(f'  Statistic : {stat:.4f}')
print(f'  p-value   : {p:.6f}')
print(f'  Skewness  : {target_log.skew():.4f}')
print(f'  Kurtosis   : {target_log.kurt():.4f}')
print(f'\nNote: On {len(target_log):,} rows, D Agostino will almost al
print('Trust the QQ plot more than the p-value on large datasets.')
```



D Agostino K2 Normality Test:

```
Statistic : 364.8956
p-value   : 0.000000
Skewness  : -0.3001
Kurtosis   : -0.3715
```

Note: On 15,740 rows, D Agostino will almost always reject normality. Trust the QQ plot more than the p-value on large datasets.

## Step 2 — Feature Distributions (Training Data)

We look at the raw distributions BEFORE any transformation. This tells us which columns are skewed and need log transform.

```
In [8]: num_cols_train = X_train.select_dtypes(include=[np.number]).columns.to
fig, axes = plt.subplots(3, 3, figsize=(16, 12))
axes = axes.flatten()

for i, col in enumerate(num_cols_train):
    axes[i].hist(X_train[col], bins=40, color='steelblue', edgecolor='
    axes[i].set_title(f'{col}\nskew={X_train[col].skew():.2f}', fontsi

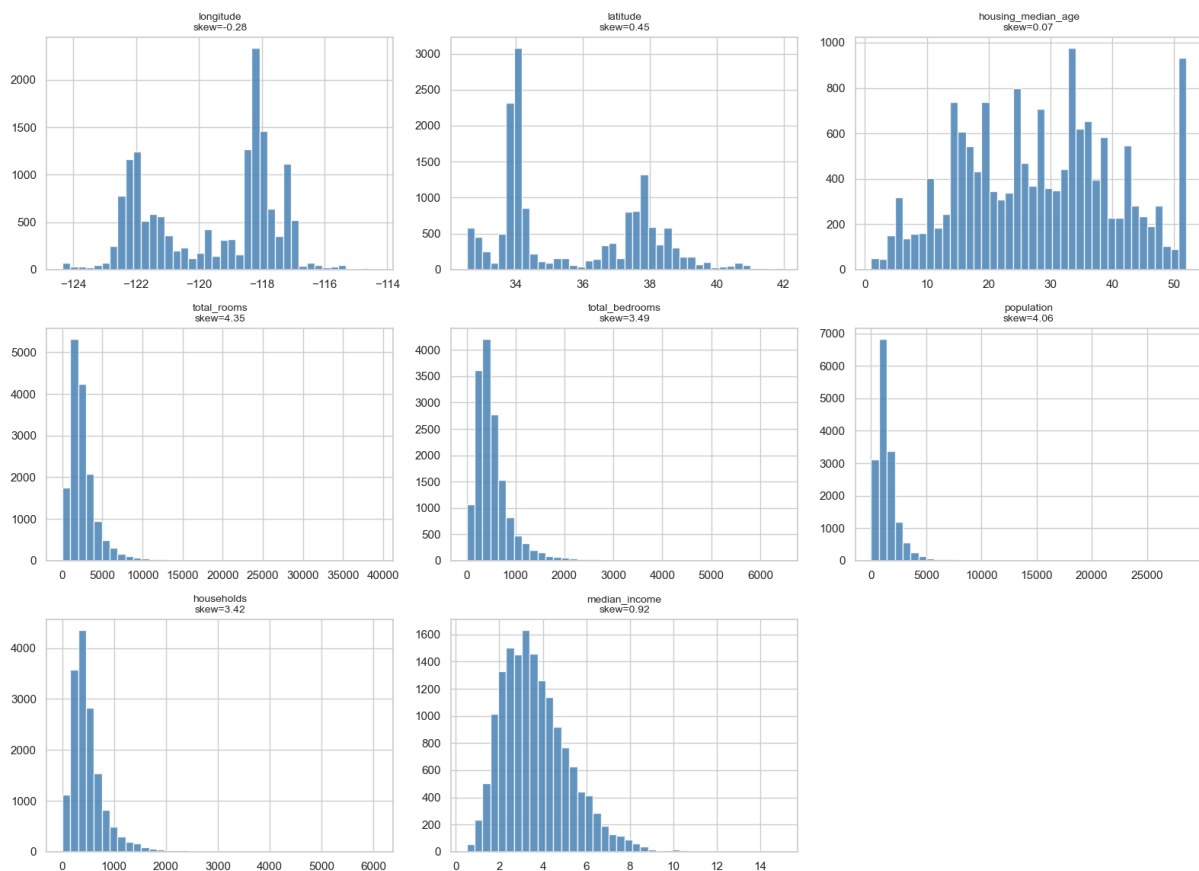
for j in range(i+1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Feature Distributions — Training Data (Raw, Before Trans
plt.tight_layout()
plt.show()

print('Skewness per column (>1 = right-skewed, needs log transform):')
print(X_train.select_dtypes(include=[np.number]).skew().sort_values(as
```



Feature Distributions — Training Data (Raw, Before Transformation)



Skewness per column (>1 = right-skewed, needs log transform):

```
total_rooms      4.350
population       4.060
total_bedrooms   3.490
households       3.420
median_income    0.920
latitude         0.450
housing_median_age 0.070
longitude        -0.280
dtype: float64
```

## Step 3 — Correlation Heatmap (Training Data)

**Pearson correlation** measures linear relationship between two variables. We compute this on training data only to understand which features correlate with the target and which features correlate with each other (multicollinearity).

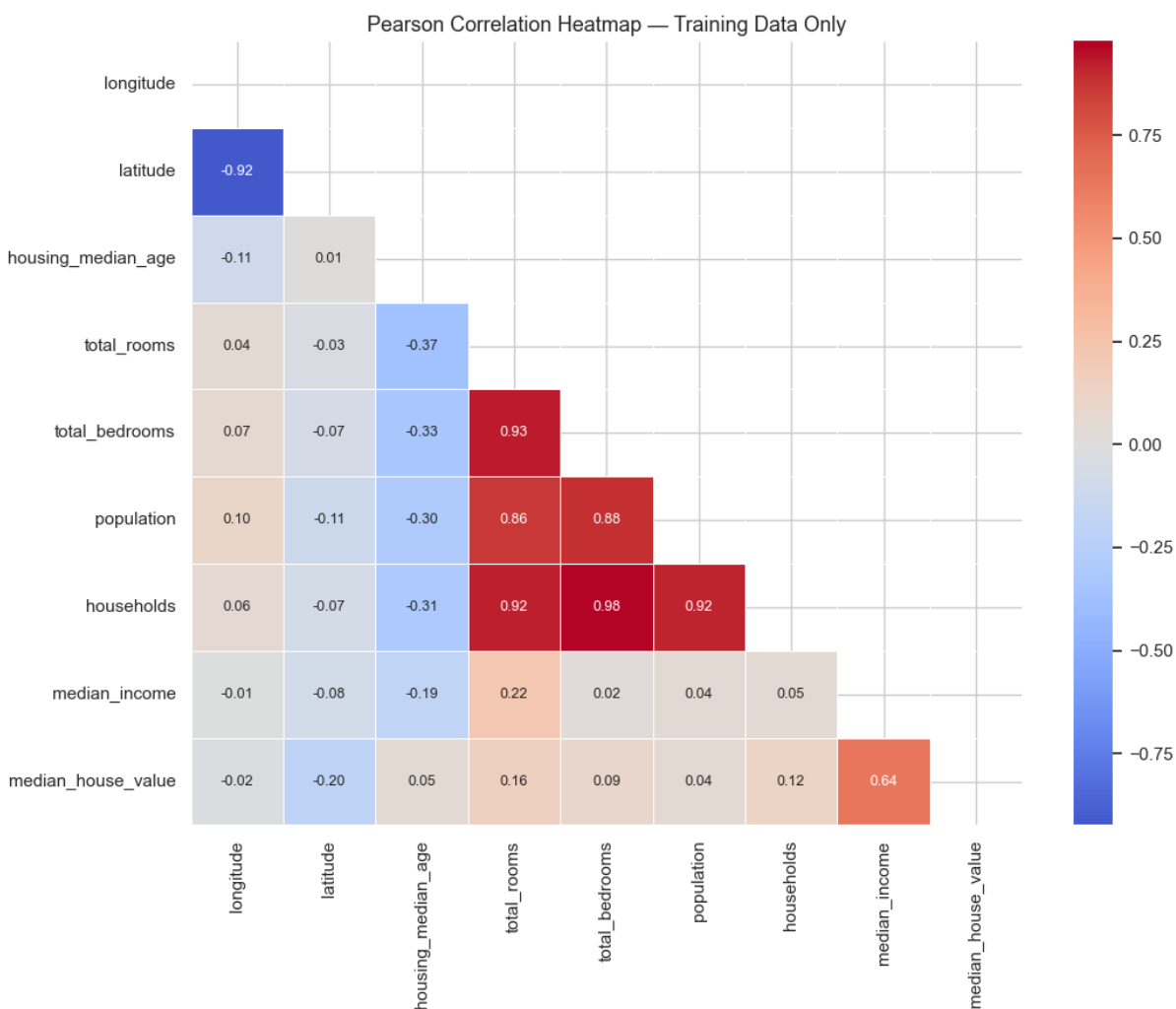
**Note:** We join `X_train` and `y_train_log` here to see target correlations.

```
In [9]: train_df = X_train.copy()
        train_df['median_house_value'] = y_train_log

        num_train = train_df.select_dtypes(include=[np.number])
        corr = num_train.corr()
```

```
fig, ax = plt.subplots(figsize=(11, 9))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, ax=ax, annot_kws={'size': 9})
ax.set_title('Pearson Correlation Heatmap - Training Data Only', fontsize=9)
plt.tight_layout()
plt.show()

print('Correlation with target (sorted):')
print(corr['median_house_value'].drop('median_house_value').sort_values)
```



```
Correlation with target (sorted):
median_income      0.638
total_rooms        0.156
households         0.118
total_bedrooms     0.094
housing_median_age 0.045
population         0.044
longitude          -0.018
latitude           -0.197
Name: median_house_value, dtype: float64
```

## Step 4 — Feature vs Target Scatter Plots (Training Data)

### What each scatter pattern means:

- Diagonal band: strong linear relationship — linear models will work well
- Curve: non-linear relationship — need tree-based models or polynomial features
- Random cloud: no signal — candidate for removal
- Funnel shape: heteroscedasticity — variance of errors grows with prediction value

**Key observation from scatter plots:** `latitude` and `longitude` individually show scattered, non-linear patterns. But we know location drives price — we just need to combine them into a meaningful distance feature.

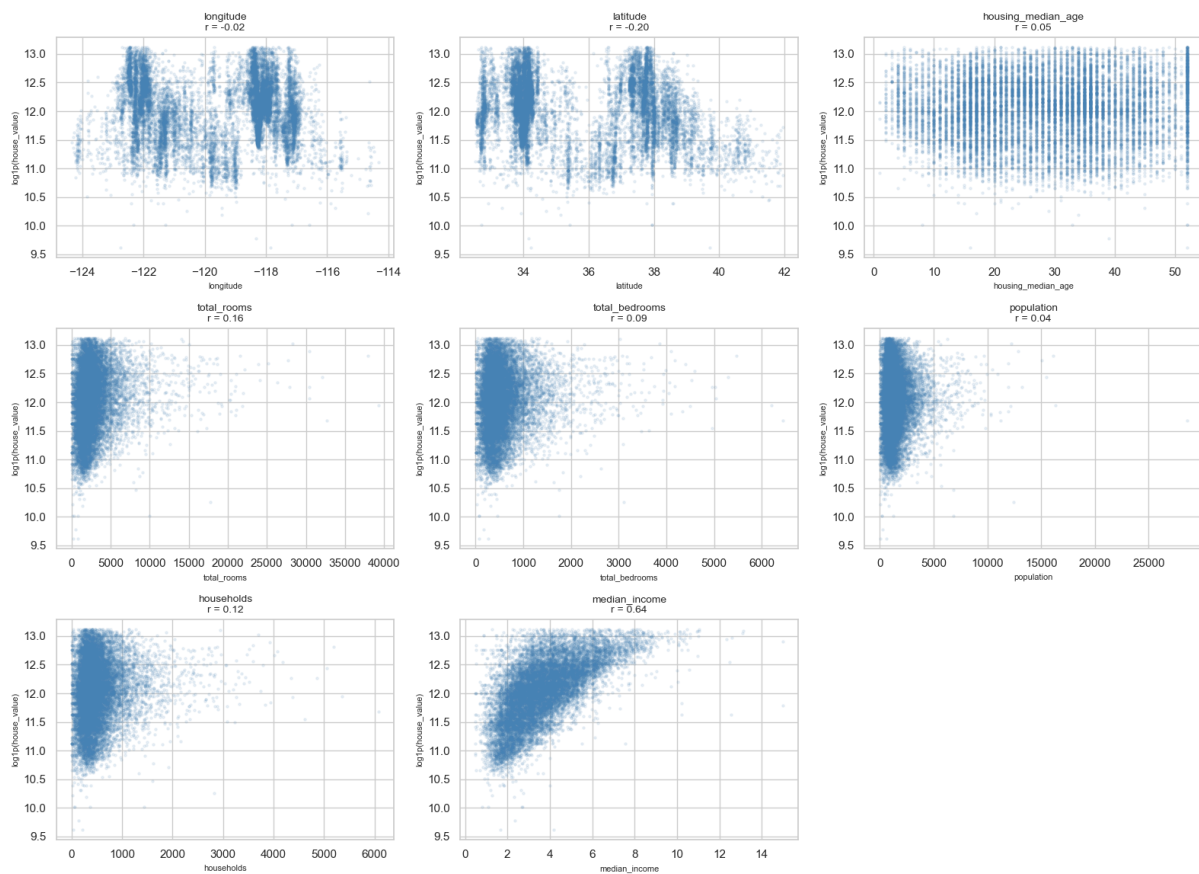
```
In [10]: feature_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
fig, axes = plt.subplots(3, 3, figsize=(16, 12))
axes = axes.flatten()

for i, col in enumerate(feature_cols):
    axes[i].scatter(X_train[col], y_train_log, alpha=0.1, s=5, color='r')
    r = X_train[col].corr(y_train_log)
    axes[i].set_title(f'{col}\nr = {r:.2f}', fontsize=10)
    axes[i].set_xlabel(col, fontsize=8)
    axes[i].set_ylabel('log1p(house_value)', fontsize=8)

for j in range(i+1, len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Feature vs Target Scatter Plots – Training Data Only', f
plt.tight_layout()
plt.show()
```

Feature vs Target Scatter Plots — Training Data Only



## Step 5 — Geographical Analysis (Training Data)

Scatter plot of lat/lon colored by house value and income. This confirms that location clusters (Bay Area, LA) drive price — motivating the `dist_to_nearest_hub` feature we will engineer in the pipeline.

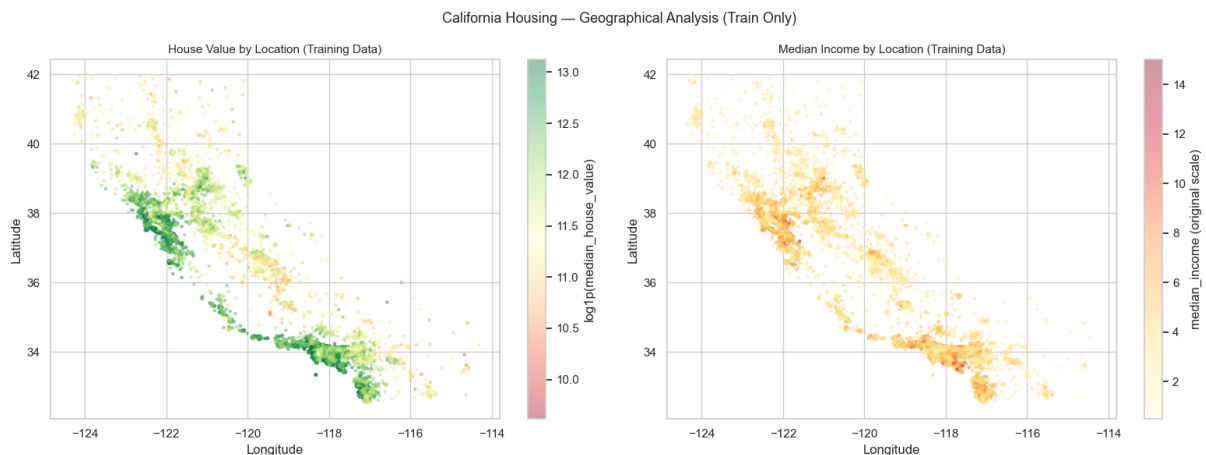
```
In [11]: fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sc = axes[0].scatter(X_train['longitude'], X_train['latitude'],
                    c=y_train_log, cmap='RdYlGn', alpha=0.4, s=5)
plt.colorbar(sc, ax=axes[0], label='log1p(median_house_value)')
axes[0].set_title('House Value by Location (Training Data)', fontsize=
axes[0].set_xlabel('Longitude')
axes[0].set_ylabel('Latitude')

sc2 = axes[1].scatter(X_train['longitude'], X_train['latitude'],
                    c=X_train['median_income'], cmap='YlOrRd', alpha=
plt.colorbar(sc2, ax=axes[1], label='median_income (original scale)')
axes[1].set_title('Median Income by Location (Training Data)', fontsize=
axes[1].set_xlabel('Longitude')
axes[1].set_ylabel('Latitude')

plt.suptitle('California Housing – Geographical Analysis (Train Only)')
plt.tight_layout()
```

```
plt.show()
```



## Step 6 — Ocean Proximity: Boxplot + ANOVA + Effect Size (Training Data)

**ANOVA tests:** Do ocean\_proximity group means differ significantly?

**Eta squared ( $\eta^2$ ) measures effect size:** How much of the total variance does ocean\_proximity explain?

$$\eta^2 = SS_{\text{between\_groups}} / SS_{\text{total}}$$

| $\eta^2$    | Interpretation |
|-------------|----------------|
| < 0.01      | Trivial        |
| 0.01 – 0.06 | Small          |
| 0.06 – 0.14 | Medium         |
| > 0.14      | Large          |

**Why p-value alone is not enough:** With 15,000+ training rows, almost anything gives  $p < 0.05$ . Effect size tells you whether the difference is PRACTICALLY meaningful.

```
In [12]: train_full = X_train.copy()
train_full['median_house_value'] = y_train_log

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
order = train_full.groupby('ocean_proximity')['median_house_value'].me

sns.boxplot(data=train_full, x='ocean_proximity', y='median_house_valu
            order=order, palette='muted', ax=axes[0])
axes[0].set_title('House Value by Ocean Proximity (Training Data)')
axes[0].set_ylabel('log1p(median_house_value)')
axes[0].tick_params(axis='x', rotation=30)
```

```

sns.violinplot(data=train_full, x='ocean_proximity', y='median_house_v
                order=order, palette='muted', ax=axes[1], inner='quarti
axes[1].set_title('Violin Plot (Training Data)')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()

# ANOVA + Eta squared
groups = [g['median_house_value'].values for _, g in
          train_full.groupby('ocean_proximity', observed=False)]
f_stat, p_val = stats.f_oneway(*groups)
grand_mean = train_full['median_house_value'].mean()
ss_total = ((train_full['median_house_value'] - grand_mean)**2).sum()
ss_between = sum(len(g)*(g.mean()-grand_mean)**2 for g in groups)
eta_sq = ss_between / ss_total

print(f'ANOVA: F={f_stat:.2f}, p={p_val:.2e}')
print(f'Eta squared (effect size): {eta_sq:.4f} -> ocean_proximity exp
print('-> Large effect (> 0.14)')

```

```

/var/folders/mw/jbc41xx11gq3v2wqc8yz62400000gn/T/ipykernel_23542/393733
0287.py:5: FutureWarning: The default of observed=False is deprecated a
nd will be changed to True in a future version of pandas. Pass observed
=False to retain current behavior or observed=True to adopt the future
default and silence this warning.

```

```

    order = train_full.groupby('ocean_proximity')['median_house_value'].m
edian().sort_values(ascending=False).index
/var/folders/mw/jbc41xx11gq3v2wqc8yz62400000gn/T/ipykernel_23542/393733
0287.py:7: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.boxplot(data=train_full, x='ocean_proximity', y='median_house_val
ue',
/var/folders/mw/jbc41xx11gq3v2wqc8yz62400000gn/T/ipykernel_23542/393733
0287.py:13: FutureWarning:

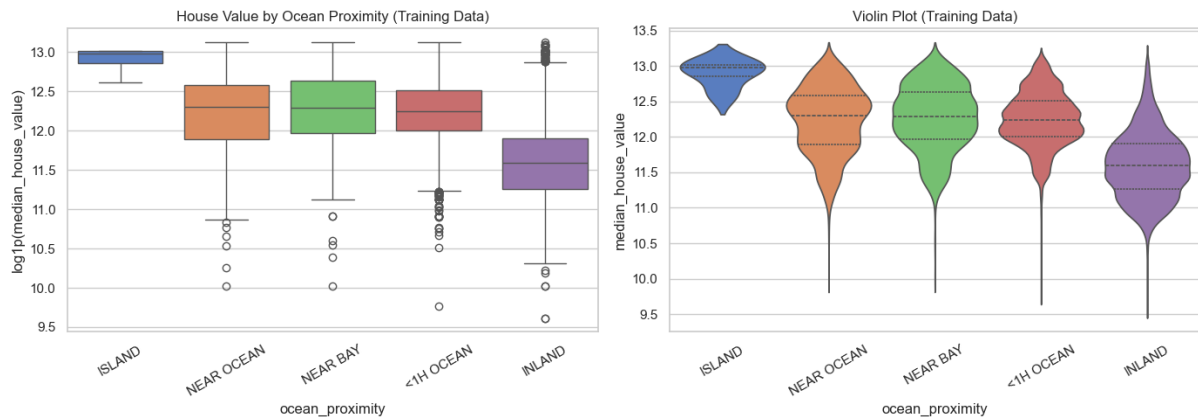
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.violinplot(data=train_full, x='ocean_proximity', y='median_house_
value',

```



ANOVA:  $F=1882.10$ ,  $p=0.00e+00$

Eta squared (effect size): 0.3236 → ocean\_proximity explains 32.4% of variance

→ Large effect ( $> 0.14$ )

## Step 7 — Spearman Correlation with Effect Size (Training Data)

**Spearman  $\rho$**  captures monotonic relationships — it works even when the relationship is non-linear as long as it is consistently increasing or decreasing.

Unlike Pearson, Spearman  $\rho$  IS its own effect size:

| $ \rho $      | Interpretation |
|---------------|----------------|
| $< 0.10$      | Trivial        |
| $0.10 - 0.30$ | Small          |
| $0.30 - 0.50$ | Medium         |
| $> 0.50$      | Large          |

```
In [13]: feat_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
spearman_rows = []
for col in feat_cols:
    rho, p = stats.spearmanr(X_train[col], y_train_log)
    eff = ('Trivial' if abs(rho)<0.10 else 'Small' if abs(rho)<0.30
          else 'Medium' if abs(rho)<0.50 else 'Large')
    spearman_rows.append({'Feature':col, 'Spearman rho':round(rho,3),
                          'p-value':f'{p:.2e}', 'Effect':eff})

sp_df = (pd.DataFrame(spearman_rows).set_index('Feature')
         .sort_values('Spearman rho', key=abs, ascending=False))
print('Spearman Correlation with target (training data only):')
print(sp_df)
```

Spearman Correlation with target (training data only):

|                    | Spearman rho | p-value   | Effect  |
|--------------------|--------------|-----------|---------|
| Feature            |              |           |         |
| median_income      | 0.649        | 0.00e+00  | Large   |
| total_rooms        | 0.203        | 8.45e-146 | Small   |
| latitude           | -0.171       | 8.12e-104 | Small   |
| households         | 0.133        | 1.15e-62  | Small   |
| longitude          | -0.059       | 1.29e-13  | Trivial |
| housing_median_age | 0.050        | 3.78e-10  | Trivial |
| population         | 0.035        | 1.20e-05  | Trivial |
| total_bedrooms     | NaN          | nan       | Large   |

## Step 8 — VIF Analysis (Training Data)

**VIF (Variance Inflation Factor)** measures how much a feature's coefficient is inflated by its correlation with other features — NOT by its relationship with the target.

Formula:

VIF for Feature X =  $1 / (1 - R^2)$   
 where  $R^2$  = how well X can be predicted by ALL other features combined

| VIF    | Status | Action                            |
|--------|--------|-----------------------------------|
| 1 – 5  | Clean  | Keep                              |
| 5 – 10 | High   | Investigate                       |
| > 10   | Severe | Drop, combine, or use Ridge/Lasso |

**High VIF ≠ bad for prediction.** It means: for linear models, coefficients will be unstable. Tree models are largely unaffected by multicollinearity.

We compute VIF on training data only — this is EDA to inform our feature engineering decisions.

```
In [14]: from statsmodels.tools.tools import add_constant
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
vif_input = X_train[vif_cols].dropna()
X_vif = add_constant(vif_input)

vif_df = pd.DataFrame({
    'Feature': vif_cols,
    'VIF': [round(variance_inflation_factor(X_vif.values, i+1), 2)
            for i in range(len(vif_cols))]
```



```

}).set_index('Feature').sort_values('VIF', ascending=False)

vif_df['Status'] = vif_df['VIF'].apply(
    lambda v: 'Clean' if v<5 else 'High' if v<10 else 'Severe')

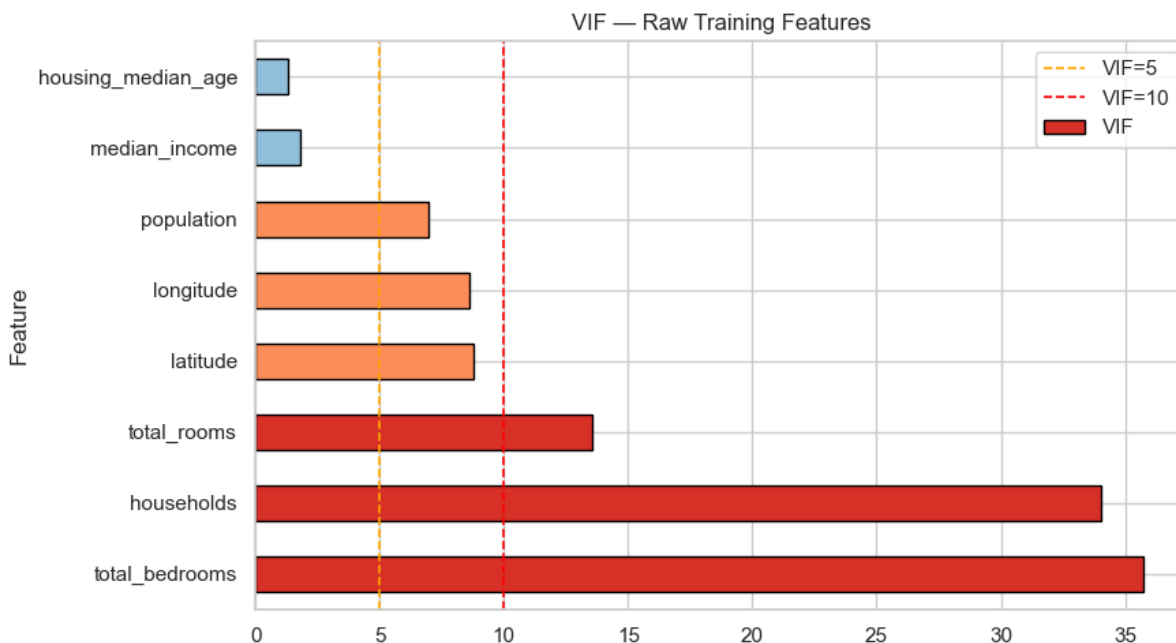
print('VIF on raw training features:')
print(vif_df)

fig, ax = plt.subplots(figsize=(9, 5))
colors = ['#d73027' if v>10 else '#fc8d59' if v>5 else '#91bfdb'
          for v in vif_df['VIF']]
vif_df['VIF'].plot(kind='barh', ax=ax, color=colors, edgecolor='black')
ax.axvline(5, color='orange', linestyle='--', linewidth=1.2, label='V
ax.axvline(10, color='red', linestyle='--', linewidth=1.2, label='V
ax.set_title('VIF - Raw Training Features')
ax.legend()
plt.tight_layout()
plt.show()

```

VIF on raw training features:

|                    | VIF    | Status |
|--------------------|--------|--------|
| Feature            |        |        |
| total_bedrooms     | 35.760 | Severe |
| households         | 34.060 | Severe |
| total_rooms        | 13.530 | Severe |
| latitude           | 8.760  | High   |
| longitude          | 8.590  | High   |
| population         | 6.950  | High   |
| median_income      | 1.780  | Clean  |
| housing_median_age | 1.300  | Clean  |



## EDA Summary — Feature Engineering Decisions

Based on training data analysis:

**Transformations needed:**

- `total_bedrooms` : 207 missing values — grouped median imputation by `ocean_proximity`
- `total_rooms` , `total_bedrooms` , `population` , `households` , `median_income` : right-skewed — log1p transform
- Count columns (`total_rooms`, `total_bedrooms`, `households`, `population`): severe VIF — replace with ratios
- `latitude` , `longitude` : weak individual signal — combine into `dist_to_nearest_hub`
- `ocean_proximity` : categorical — one-hot encode

**Feature engineering plan:**

1. Impute missing `total_bedrooms` by `ocean_proximity` group median (learned from train)
2. Winsorize numerical columns at IQR bounds (bounds learned from train)
3. Log1p transform: `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`
4. Add distance features: `dist_to_sf`, `dist_to_la`, `dist_to_sd`, `dist_to_nearest_hub`
5. Add ratio features: `rooms_per_household`, `bedrooms_per_room`, `population_per_household`
6. Drop: `latitude`, `longitude`, `dist_to_sf`, `dist_to_la`, `dist_to_sd`, `total_rooms`, `total_bedrooms`, `households`, `population`
7. One-hot encode `ocean_proximity` (categories learned from train)

All of steps 1, 2, 7 require fitting — they will be fitted on `X_train` only. Steps 3, 4, 5, 6 are deterministic — they can be applied the same way to any data.

---

## Chunk 4: Custom Sklearn Transformers

**The sklearn Transformer pattern:**

All sklearn transformers follow two-step protocol:

- `fit(X_train)` : learn from training data (compute medians, IQR bounds, categories, etc.)
- `transform(X)` : apply the learned transformation to any dataset

By inheriting from `BaseEstimator` and `TransformerMixin` :

- `BaseEstimator` gives us `get_params()` and `set_params()` — needed for `GridSearch` / `RandomizedSearch`
- `TransformerMixin` gives us `fit_transform()` for free — it just calls `fit()` then `transform()`

```
class MyTransformer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        # Learn from X – compute and store whatever statistics
you need
        # y=None is standard convention (transformers usually
ignore the target)
        return self # ALWAYS return self from fit()

    def transform(self, X):
        # Apply the transformation – use ONLY what was stored
during fit()
        # Never compute new statistics here – that would be
leakage
        return X_transformed
```

Each transformer below follows this exact pattern.

```
In [15]: from sklearn.base import BaseEstimator, TransformerMixin

# — Transformer 1: Grouped Median Imputer —
class GroupedMedianImputer(BaseEstimator, TransformerMixin):
    """
    Fills missing values in target_col using the median per group_col
    fit()      : computes group medians on training data
    transform : fills missing values using THOSE fitted medians (not r
    """
    def __init__(self, group_col='ocean_proximity', target_col='total_
        self.group_col = group_col
        self.target_col = target_col

    def fit(self, X, y=None):
        # Learn group medians from training data only
        self.group_medians_ = (X.groupby(self.group_col, observed=True)
                               .median().to_dict())
        # Fallback: global median from training data (for unseen categ
        self.global_median_ = X[self.target_col].median()
        return self

    def transform(self, X):
        X = X.copy()
        for group, median in self.group_medians_.items():
            mask = (X[self.group_col] == group) & (X[self.target_col].
                X.loc[mask, self.target_col] = median
        # Catch remaining NaN (e.g., unseen category in test)
        X[self.target_col] = X[self.target_col].fillna(self.global_med
        return X
```

```
print('GroupedMedianImputer defined.')
```

GroupedMedianImputer defined.

```
In [16]: # — Transformer 2: Winsorizer —————
class Winsorizer(BaseEstimator, TransformerMixin):
    """
    Clips numerical columns to IQR-based bounds.
    fit()      : computes Q1, Q3, IQR and clip bounds on training data
    transform  : clips values using those FITTED bounds (not recomputed)

    Why this matters: bounds learned from train are applied to test.
    A test row with value 39,000 gets clipped to the same bound as tra
    This is what would happen with real new data in production.
    """
    def __init__(self, cols):
        self.cols = cols

    def fit(self, X, y=None):
        self.bounds_ = {}
        for col in self.cols:
            Q1 = X[col].quantile(0.25)
            Q3 = X[col].quantile(0.75)
            IQR = Q3 - Q1
            self.bounds_[col] = (Q1 - 1.5*IQR, Q3 + 1.5*IQR)
        return self

    def transform(self, X):
        X = X.copy()
        for col in self.cols:
            lo, hi = self.bounds_[col]
            X[col] = X[col].clip(lower=lo, upper=hi)
        return X

print('Winsorizer defined.')
```

Winsorizer defined.

```
In [17]: # — Transformer 3: Log Transformer —————
class LogTransformer(BaseEstimator, TransformerMixin):
    """
    Applies np.log1p() to specified columns.
    No fitting needed – log1p is a deterministic formula, learns nothing
    log1p(x) = log(x+1) – handles zero values safely (log(0) is undefi

    Why log transform?
    – StandardScaler: changes scale but NOT shape (skewness remains)
    – Log transform: changes the actual distribution shape (compresses
    – Right-skewed -> near-symmetric after log1p

    Side effect we use in Chunk 5:
    In log space, subtraction = log ratio.
```

```

log(a) - log(b) = log(a/b)
So rooms_per_household = log(total_rooms) - log(households) = log(
"""
def __init__(self, cols):
    self.cols = cols

def fit(self, X, y=None):
    return self # Nothing to learn

def transform(self, X):
    X = X.copy()
    for col in self.cols:
        X[col] = np.log1p(X[col])
    return X

print('LogTransformer defined.')

```

LogTransformer defined.

```

In [18]: # — Transformer 4: Distance Feature Adder —
class DistanceFeatureAdder(BaseEstimator, TransformerMixin):
    """
    Adds Euclidean distance features to major California city hubs.
    No fitting needed – uses fixed geographic coordinates.

    Why combine lat and lon into distance?
    - latitude=37.8 alone tells you nothing meaningful
    - longitude=-122.4 alone tells you nothing meaningful
    - Together they describe San Francisco – most expensive cluster
    - Individual correlations with target: 0.06 – 0.17
    - Combined dist_to_nearest_hub correlation: -0.52 (3-4x improvement)

    Why Euclidean and not Haversine (great-circle distance)?
    California spans only ~10 degrees lat x 10 degrees lon.
    At this scale, Euclidean error vs Haversine is under 0.5%.
    Haversine adds code complexity with no meaningful accuracy gain here
    """
    SF = (37.7749, -122.4194)
    LA = (34.0522, -118.2437)
    SD = (32.7157, -117.1611)

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        def edist(lat, lon, rlat, rlon):
            return np.sqrt((lat - rlat)**2 + (lon - rlon)**2)
        X['dist_to_sf'] = edist(X['latitude'], X['longitude'], *self.SF)
        X['dist_to_la'] = edist(X['latitude'], X['longitude'], *self.LA)
        X['dist_to_sd'] = edist(X['latitude'], X['longitude'], *self.SD)
        X['dist_to_nearest_hub'] = X[['dist_to_sf', 'dist_to_la', 'dist_
        return X

```

```
print('DistanceFeatureAdder defined.')
```

DistanceFeatureAdder defined.

```
In [19]: # — Transformer 5: Ratio Feature Adder —
class RatioFeatureAdder(BaseEstimator, TransformerMixin):
    """
    Creates ratio features from count columns (which are already in log space)
    No fitting needed – pure arithmetic.

    Why ratios beat raw counts:
    - total_rooms=10,000 in a district with households=5,000 -> 2 room
    - total_rooms=10,000 in a district with households=1,000 -> 10 room
    - The raw count 10,000 tells you nothing. The ratio does.

    Since columns are in log space (log1p applied before this step):
    log(total_rooms) - log(households) = log(total_rooms / households)
    Subtraction in log space = division in original space.
    """
    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X['rooms_per_household'] = X['total_rooms'] - X['total_households']
        X['bedrooms_per_room'] = X['total_bedrooms'] - X['total_rooms']
        X['population_per_household'] = X['population'] - X['total_households']
        return X

print('RatioFeatureAdder defined.')
```

RatioFeatureAdder defined.

```
In [20]: # — Transformer 6: Column Dropper —
class ColumnDropper(BaseEstimator, TransformerMixin):
    """
    Drops specified columns.
    No fitting needed.

    We drop:
    - latitude, longitude: summarized by dist_to_nearest_hub
    - dist_to_sf, dist_to_la, dist_to_sd: summarized by dist_to_nearest_city
    - total_rooms, total_bedrooms, households, population:
      replaced by ratio features (rooms_per_household etc.)
      keeping them alongside ratios would reintroduce the same multicollinearity
    """
    def __init__(self, cols):
        self.cols = cols

    def fit(self, X, y=None):
        return self
```

```
def transform(self, X):
    return X.drop(columns=self.cols, errors='ignore')

print('ColumnDropper defined.')
```

ColumnDropper defined.

```
In [21]: # — Transformer 7: Ocean Proximity Encoder —————
class OceanProximityEncoder(BaseEstimator, TransformerMixin):
    """
    One-hot encodes ocean_proximity using categories learned from train
    fit()      : records unique categories in training data
    transform : creates binary columns using those categories

    Why fit() matters here:
    If we used pd.get_dummies() on X_train and X_test separately, and
    appears in test but not train, we'd get an extra column – mismatch
    By fitting on train, we define exactly which columns to create, and
    the same column set to test regardless of what categories appear in
    test

    drop_first=True: drops one reference category to avoid dummy variable
    trap. With 5 categories and 4 dummies, if all 4 are 0 the model knows it
    belongs to the 5th category. Keeping all 5 creates perfect multicollinearity (one column = 1 -
    sum of others)

    Why NOT label encoding (0, 1, 2, 3, 4)?
    Label encoding implies order: INLAND < ISLAND < NEAR BAY ...
    There is no such ordering – these are unordered categories.
    A model using label encoding would think ISLAND is 'between' INLAND and NEAR BAY
    """
    def __init__(self, col='ocean_proximity', drop_first=True):
        self.col = col
        self.drop_first = drop_first

    def fit(self, X, y=None):
        cats = sorted(X[self.col].astype(str).unique())
        self.categories_ = cats[1:] if self.drop_first else cats
        return self

    def transform(self, X):
        X = X.copy()
        for cat in self.categories_:
            X[f'ocean_proximity_{cat}'] = (X[self.col].astype(str) == cat)
        X = X.drop(columns=[self.col])
        return X

print('OceanProximityEncoder defined.')
print('\nAll 7 custom transformers ready.')
```

OceanProximityEncoder defined.

All 7 custom transformers ready.

## Chunk 5: Build & Apply the Preprocessing Pipeline

**sklearn Pipeline chains transformers in order.** When you call `pipeline.fit(X_train)`, it calls `fit_transform()` on each step in sequence. When you call `pipeline.transform(X_test)`, it calls `transform()` on each step using the parameters learned during `fit()` on train.

```
preprocessor.fit(X_train)
    -> GroupedMedianImputer.fit(X_train) : learns group
medians from train
    -> Winsorizer.fit(X_train)           : learns IQR
bounds from train
    -> LogTransformer.fit(X_train)       : nothing to
learn
    -> DistanceFeatureAdder.fit(X_train) : nothing to
learn
    -> RatioFeatureAdder.fit(X_train)    : nothing to
learn
    -> ColumnDropper.fit(X_train)        : nothing to
learn
    -> OceanProximityEncoder.fit(X_train) : learns
categories from train

preprocessor.transform(X_test)
    -> uses ONLY what was fitted above – no information
from X_test
```

This guarantees zero data leakage.

```
In [22]: from sklearn.pipeline import Pipeline

WIN_COLS = ['longitude', 'latitude', 'housing_median_age',
            'total_rooms', 'total_bedrooms', 'population',
            'households', 'median_income']

LOG_COLS = ['total_rooms', 'total_bedrooms', 'population',
            'households', 'median_income']

DROP_COLS = ['latitude', 'longitude',
            'dist_to_sf', 'dist_to_la', 'dist_to_sd',
            'total_rooms', 'total_bedrooms', 'households', 'populatio

preprocessor = Pipeline([
    ('imputer', GroupedMedianImputer(group_col='ocean_proximity',
                                     target_col='total_bedrooms')),
    ('winsor', Winsorizer(cols=WIN_COLS)),
```



```

    ('log',      LogTransformer(cols=LOG_COLS)),
    ('distance', DistanceFeatureAdder()),
    ('ratios',   RatioFeatureAdder()),
    ('drop',     ColumnDropper(cols=DROP_COLS)),
    ('encode',   OceanProximityEncoder(col='ocean_proximity', drop_fir
])

# Fit on TRAIN only, then transform both
X_train_proc = preprocessor.fit_transform(X_train)
X_test_proc  = preprocessor.transform(X_test)

print('Preprocessor fitted on training data and applied to both sets.')
print(f'X_train_proc shape: {X_train_proc.shape}')
print(f'X_test_proc  shape: {X_test_proc.shape}')
print(f'\nFinal features ({X_train_proc.shape[1]}):')
for i, col in enumerate(X_train_proc.columns, 1):
    print(f' {i:2d}. {col}')

```

Preprocessor fitted on training data and applied to both sets.

X\_train\_proc shape: (15740, 10)

X\_test\_proc shape: (3935, 10)

Final features (10):

1. housing\_median\_age
2. median\_income
3. dist\_to\_nearest\_hub
4. rooms\_per\_household
5. bedrooms\_per\_room
6. population\_per\_household
7. ocean\_proximity\_INLAND
8. ocean\_proximity\_ISLAND
9. ocean\_proximity\_NEAR BAY
10. ocean\_proximity\_NEAR OCEAN

## Verify: What the Imputer Learned from Training Data

```

In [23]: imp = preprocessor.named_steps['imputer']
print('Group medians learned from X_train:')
for group, median in imp.group_medians_.items():
    print(f' {group:15s}: {median:.1f}')
print(f' Global fallback : {imp.global_median_:.1f}')

win = preprocessor.named_steps['winsor']
print('\nWinsorization bounds learned from X_train (sample):')
for col in ['total_rooms', 'total_bedrooms', 'population']:
    lo, hi = win.bounds_[col]
    print(f' {col:20s}: [{lo:.1f}, {hi:.1f}']')

enc = preprocessor.named_steps['encode']
print('\nOne-hot categories learned from X_train:')
print(' ', enc.categories_)

```

Group medians learned from X\_train:

```
<1H OCEAN      : 442.0
INLAND         : 424.0
ISLAND         : 516.5
NEAR BAY       : 426.0
NEAR OCEAN     : 474.5
Global fallback : 438.0
```

Winsorization bounds learned from X\_train (sample):

```
total_rooms      : [-1076.0, 5636.0]
total_bedrooms   : [-222.5, 1165.5]
population       : [-620.5, 3167.5]
```

One-hot categories learned from X\_train:

```
['INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN']
```

## Post-Pipeline VIF Check

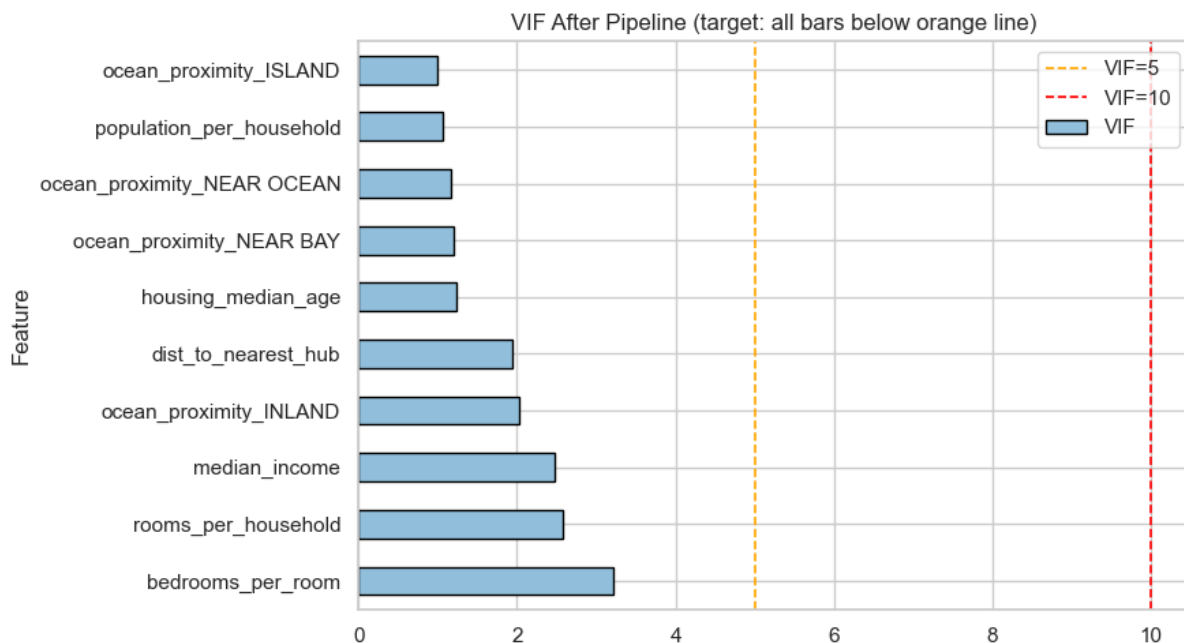
```
In [24]: vif_input2 = X_train_proc.select_dtypes(include=[np.number])
X_vif2 = add_constant(vif_input2)
vif2 = pd.DataFrame({
    'Feature': vif_input2.columns,
    'VIF': [round(variance_inflation_factor(X_vif2.values, i+1), 2)
            for i in range(len(vif_input2.columns))]
}).set_index('Feature').sort_values('VIF', ascending=False)
vif2['Status'] = vif2['VIF'].apply(lambda v: 'Clean' if v<5 else 'High')

print('VIF after preprocessing pipeline (all should be < 5):')
print(vif2)

fig, ax = plt.subplots(figsize=(9, 5))
colors = ['#d73027' if v>10 else '#fc8d59' if v>5 else '#91bfdb' for v in vif2['VIF']]
vif2['VIF'].plot(kind='barh', ax=ax, color=colors, edgecolor='black')
ax.axvline(5, color='orange', linestyle='--', linewidth=1.2, label='VIF=5')
ax.axvline(10, color='red', linestyle='--', linewidth=1.2, label='VIF=10')
ax.set_title('VIF After Pipeline (target: all bars below orange line)')
ax.legend()
plt.tight_layout()
plt.show()
```

VIF after preprocessing pipeline (all should be < 5):

| Feature                    | VIF   | Status |
|----------------------------|-------|--------|
| bedrooms_per_room          | 3.220 | Clean  |
| rooms_per_household        | 2.580 | Clean  |
| median_income              | 2.470 | Clean  |
| ocean_proximity_INLAND     | 2.020 | Clean  |
| dist_to_nearest_hub        | 1.940 | Clean  |
| housing_median_age         | 1.240 | Clean  |
| ocean_proximity_NEAR BAY   | 1.200 | Clean  |
| ocean_proximity_NEAR OCEAN | 1.160 | Clean  |
| population_per_household   | 1.060 | Clean  |
| ocean_proximity_ISLAND     | 1.000 | Clean  |



## Chunk 6: ML Models

All models trained on `X_train_proc` / `y_train_log` and evaluated on `X_test_proc` / `y_test_log`.

**Models (simple to complex):**

1. Linear Regression — baseline, most interpretable
2. Ridge Regression — L2 regularization
3. Lasso Regression — L1 regularization, automatic feature selection
4. Random Forest — ensemble trees, captures non-linearity
5. XGBoost — gradient boosting, typically strongest on tabular data

**Metrics reported in log AND dollar scale.** Dollar scale:

`np.expm1(prediction)` reverses the log transform.

```
In [25]: from sklearn.metrics import mean_squared_error, mean_absolute_error, r

results_v2 = {}

def evaluate(name, model, X_tr, y_tr, X_te, y_te_log):
    model.fit(X_tr, y_tr)
    pred_log = model.predict(X_te)
    pred_dollar = np.expm1(pred_log)
    y_dollar = np.expm1(y_te_log)

    r2 = r2_score(y_te_log, pred_log)
    rmse_log = np.sqrt(mean_squared_error(y_te_log, pred_log))
    mae_log = mean_absolute_error(y_te_log, pred_log)
    rmse_dollar = np.sqrt(mean_squared_error(y_dollar, pred_dollar))
    mae_dollar = mean_absolute_error(y_dollar, pred_dollar)

    results_v2[name] = {
        'R2': round(r2, 4), 'RMSE (log)': round(rmse_log, 4),
        'MAE (log)': round(mae_log, 4),
        'RMSE ($)': round(rmse_dollar, 0), 'MAE ($)': round(mae_dollar, 0)
    }
    print(f"\n{'='*55}\n {name}\n{'='*55}")
    print(f' R2      : {r2:.4f} ({r2*100:.1f}% variance explained)')
    print(f' RMSE $ : ${rmse_dollar:,.0f}')
    print(f' MAE $ : ${mae_dollar:,.0f}')
    return model, pred_log, pred_dollar

print('Evaluation function ready.')
```

Evaluation function ready.

## Model 1 — Linear Regression (Baseline)

Fits a hyperplane by minimizing sum of squared errors:  $y_{\text{hat}} = B_0 + B_1x_1 + \dots + B_nx_n$

**Assumptions:** linear relationship, no severe multicollinearity (fixed by VIF engineering), normally distributed residuals, homoscedasticity.

**Coefficients are directly interpretable:** if `median_income` coefficient = 0.93, then a 1-unit increase in  $\log(\text{income})$  predicts a 0.93-unit increase in  $\log(\text{house\_value})$ .

**Expected R2: 0.65–0.69.** Limited by the non-linear patterns in the data that a straight line cannot capture.

```
In [26]: from sklearn.linear_model import LinearRegression

lr, lr_pred_log, lr_pred_dollar = evaluate(
```

```

    'Linear Regression', LinearRegression(),
    X_train_proc, y_train_log, X_test_proc, y_test_log
)

coef = pd.DataFrame({'Coef': lr.coef_}, index=X_train_proc.columns).so
print('\nCoefficients:')
print(coef.round(4))

```

```

=====
Linear Regression
=====
R2      : 0.6856 (68.6% variance explained)
RMSE $  : $61,579
MAE $   : $43,582

```

Coefficients:

|                            | Coef   |
|----------------------------|--------|
| median_income              | 0.935  |
| ocean_proximity_ISLAND     | 0.733  |
| bedrooms_per_room          | 0.205  |
| rooms_per_household        | 0.101  |
| ocean_proximity_NEAR OCEAN | 0.027  |
| housing_median_age         | 0.001  |
| ocean_proximity_NEAR BAY   | -0.048 |
| dist_to_nearest_hub        | -0.142 |
| ocean_proximity_INLAND     | -0.331 |
| population_per_household   | -0.386 |

## Model 2 — Ridge Regression (L2 Regularization)

Loss =  $\sum(y - \hat{y})^2 + \alpha * \sum(\beta^2)$

Ridge shrinks ALL coefficients toward zero but never to exactly zero. Handles mild remaining multicollinearity by distributing weight more evenly.

**RidgeCV** tries multiple alpha values with 5-fold CV on training data only. The test set is never touched during alpha selection.

If best alpha is very small (0.01), it means our feature engineering already handled multicollinearity and Ridge has almost nothing extra to do.

```

In [27]: from sklearn.linear_model import RidgeCV

ridge_cv = RidgeCV(alphas=[0.01, 0.1, 1.0, 10.0, 100.0, 500.0, 1000.0])
ridge, ridge_pred_log, ridge_pred_dollar = evaluate(
    'Ridge Regression', ridge_cv,
    X_train_proc, y_train_log, X_test_proc, y_test_log
)
print(f'\nBest alpha: {ridge_cv.alpha_}')

```

```
=====
Ridge Regression
=====
```

```
R2      : 0.6856 (68.6% variance explained)
RMSE $  : $61,579
MAE $   : $43,582
```

Best alpha: 0.01

## Model 3 — Lasso Regression (L1 Regularization)

Loss =  $\sum(y - \hat{y})^2 + \alpha * \sum(|\beta|)$

Lasso drives weak coefficients to EXACTLY zero — automatic feature selection.

**Geometric reason:** The L1 penalty region is a diamond shape. The optimal point often lands exactly at a corner of the diamond, where one coordinate = 0.

**If Lasso zeroes a feature:** that feature adds no predictive value beyond what others already provide. With 0 features zeroed, all 10 features carry genuine independent signal.

**LassoCV** finds best alpha via cross-validation on training data.

```
In [28]: from sklearn.linear_model import LassoCV

lasso_cv = LassoCV(alphas=[0.0001, 0.001, 0.01, 0.1, 1.0], cv=5, max_i
lasso, lasso_pred_log, lasso_pred_dollar = evaluate(
    'Lasso Regression', lasso_cv,
    X_train_proc, y_train_log, X_test_proc, y_test_log
)
print(f'\nBest alpha: {lasso_cv.alpha_:.6f}')
print(f'Features zeroed out: {(lasso_cv.coef_ == 0).sum()} / {len(lasso_cv.coef_)}')
coef_l = pd.DataFrame({'Coef': lasso_cv.coef_}, index=X_train_proc.columns)
print(coef_l.round(6))
```

```
=====
Lasso Regression
=====
R2      : 0.6856 (68.6% variance explained)
RMSE $  : $61,583
MAE $   : $43,584

Best alpha: 0.000100
Features zeroed out: 0 / 10

              Coef
median_income      0.932
ocean_proximity_ISLAND 0.342
bedrooms_per_room   0.197
rooms_per_household 0.097
ocean_proximity_NEAR OCEAN 0.026
housing_median_age  0.001
ocean_proximity_NEAR BAY -0.046
dist_to_nearest_hub -0.142
ocean_proximity_INLAND -0.331
population_per_household -0.385
```

## Model 4 — Random Forest

Builds many decision trees, each on a bootstrap sample with random feature subsets. Final prediction = average of all trees.

### Two sources of randomness:

1. Bootstrap sampling: each tree sees ~63% of rows
2. Feature subsampling: at each split, only a random subset of features is considered

**Why averaging beats one tree:** individual tree errors are random and uncorrelated. Averaging cancels them out.

**Captures non-linear relationships** — a tree can split on 'if income > 4.5 AND near ocean -> high price'. A linear model cannot represent this interaction.

**Expected R2: 0.77–0.80.** Large jump over linear models confirms genuine non-linearity.

```
In [29]: from sklearn.ensemble import RandomForestRegressor

rf, rf_pred_log, rf_pred_dollar = evaluate(
    'Random Forest',
    RandomForestRegressor(n_estimators=200, min_samples_leaf=2, random
    X_train_proc, y_train_log, X_test_proc, y_test_log
)
```

```

fi = pd.DataFrame({'Importance': rf.feature_importances_,
                  index=X_train_proc.columns).sort_values('Importance')
print('\nFeature Importances:')
print(fi.round(4))

fig, ax = plt.subplots(figsize=(9, 5))
fi['Importance'].plot(kind='barh', ax=ax, color='steelblue', edgecolor='black')
ax.set_title('Random Forest Feature Importances')
plt.tight_layout()
plt.show()

```

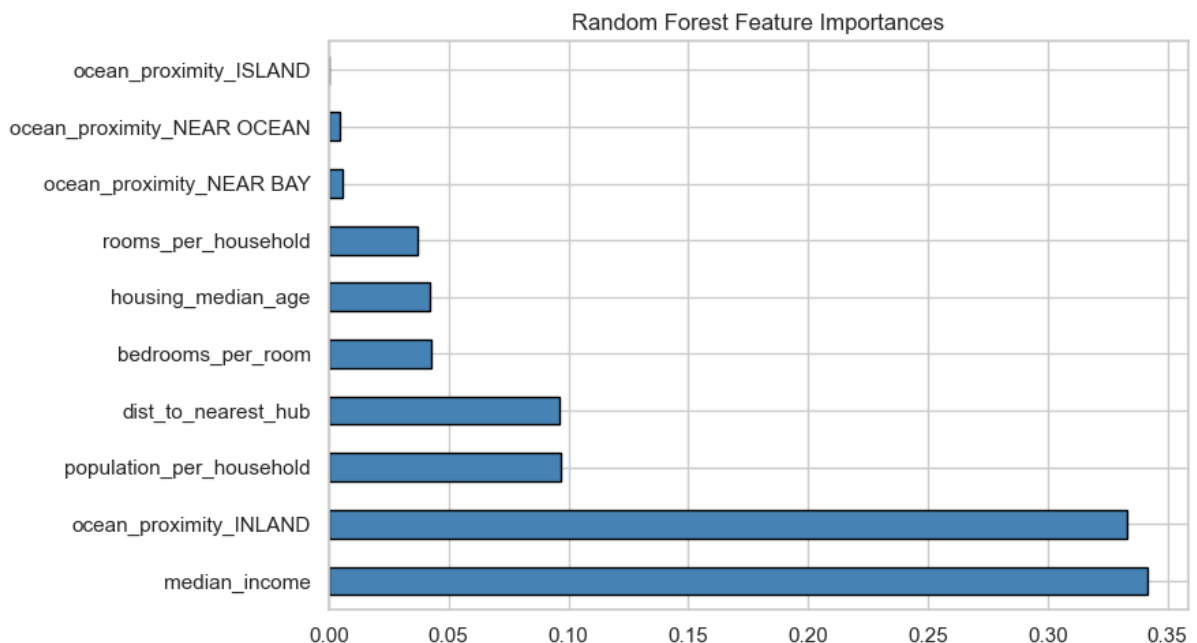
```

=====
Random Forest
=====
R2      : 0.7770 (77.7% variance explained)
RMSE $  : $52,307
MAE $   : $35,534

```

Feature Importances:

|                            | Importance |
|----------------------------|------------|
| median_income              | 0.342      |
| ocean_proximity_INLAND     | 0.333      |
| population_per_household   | 0.097      |
| dist_to_nearest_hub        | 0.096      |
| bedrooms_per_room          | 0.043      |
| housing_median_age         | 0.042      |
| rooms_per_household        | 0.037      |
| ocean_proximity_NEAR BAY   | 0.006      |
| ocean_proximity_NEAR OCEAN | 0.005      |
| ocean_proximity_ISLAND     | 0.000      |



## Model 5 — XGBoost

Builds trees sequentially — each tree corrects the errors of all previous trees.



**vs Random Forest:** RF builds trees in parallel (independent). XGBoost builds in sequence (each tree focuses on what previous trees got wrong).

### Gradient boosting steps:

1. Start with mean(y) as prediction
2. Compute residuals (errors)
3. Fit a shallow tree to predict those residuals
4. Add tree prediction  $\times$  learning\_rate to current prediction
5. Compute new residuals, repeat

**Built-in regularization:** reg\_alpha (L1) and reg\_lambda (L2) prevent overfitting.

**Expected R2: 0.78–0.83.** Usually top performer on tabular data.

```
In [30]: import xgboost as xgb

xgb_m, xgb_pred_log, xgb_pred_dollar = evaluate(
    'XGBoost',
    xgb.XGBRegressor(n_estimators=500, learning_rate=0.05, max_depth=5,
                     subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1,
                     reg_lambda=1.0, random_state=42, n_jobs=-1, verbose=0),
    X_train_proc, y_train_log, X_test_proc, y_test_log
)

fi_xgb = pd.DataFrame({'Importance': xgb_m.feature_importances_,
                      index=X_train_proc.columns).sort_values('Importance', ascending=False)
print('\nXGBoost Feature Importances:')
print(fi_xgb.round(4))

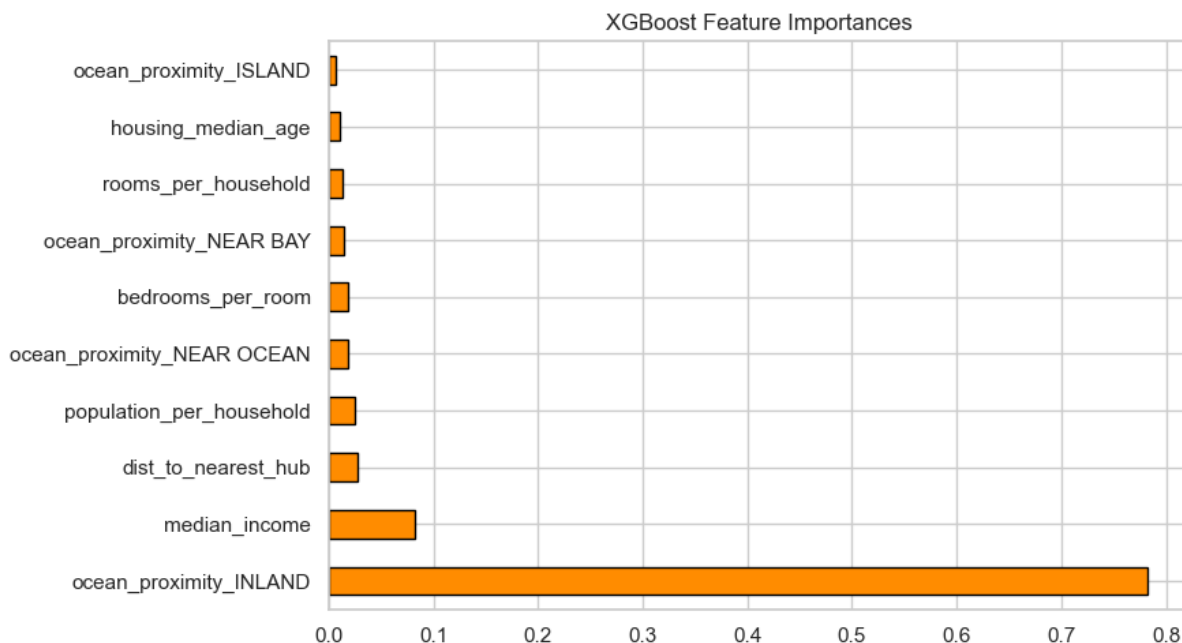
fig, ax = plt.subplots(figsize=(9, 5))
fi_xgb['Importance'].plot(kind='barh', ax=ax, color='darkorange', edgecolor='black')
ax.set_title('XGBoost Feature Importances')
plt.tight_layout()
plt.show()
```

```
=====
XGBoost
=====
```

```
R2      : 0.7823 (78.2% variance explained)
RMSE $  : $51,206
MAE $   : $34,895
```

## XGBoost Feature Importances:

|                            | Importance |
|----------------------------|------------|
| ocean_proximity_INLAND     | 0.782      |
| median_income              | 0.082      |
| dist_to_nearest_hub        | 0.027      |
| population_per_household   | 0.025      |
| ocean_proximity_NEAR OCEAN | 0.019      |
| bedrooms_per_room          | 0.018      |
| ocean_proximity_NEAR BAY   | 0.015      |
| rooms_per_household        | 0.013      |
| housing_median_age         | 0.011      |
| ocean_proximity_ISLAND     | 0.007      |



## Model Comparison — V2 Results

```
In [31]: models_preds_v2 = [
          ('Linear Regression', lr_pred_dollar),
          ('Ridge Regression',  ridge_pred_dollar),
          ('Lasso Regression',  lasso_pred_dollar),
          ('Random Forest',     rf_pred_dollar),
          ('XGBoost',           xgb_pred_dollar),
          ]
          y_test_dollar = np.expml(y_test_log)

          res_df = pd.DataFrame(results_v2).T.sort_values('R2', ascending=False)
          print('Model Comparison (V2 - Correct Pipeline):')
```

```

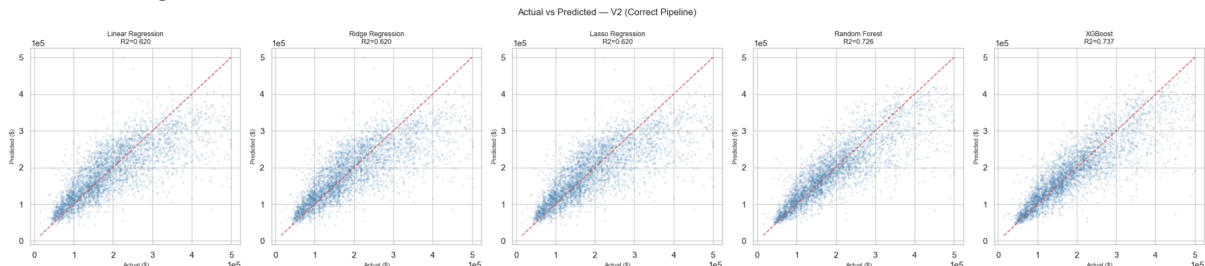
print(res_df.to_string())

# Actual vs Predicted
fig, axes = plt.subplots(1, 5, figsize=(22, 5))
for ax, (name, pred) in zip(axes, models_preds_v2):
    r2 = r2_score(y_test_dollar, pred)
    ax.scatter(y_test_dollar, pred, alpha=0.1, s=4, color='steelblue')
    lims = [y_test_dollar.min(), y_test_dollar.max()]
    ax.plot(lims, lims, 'r--', linewidth=1.2)
    ax.set_title(f'{name}\nR2={r2:.3f}', fontsize=9)
    ax.set_xlabel('Actual ($)', fontsize=8)
    ax.set_ylabel('Predicted ($)', fontsize=8)
    ax.ticklabel_format(style='sci', axis='both', scilimits=(0,0))
plt.suptitle('Actual vs Predicted - V2 (Correct Pipeline)', fontsize=12)
plt.tight_layout()
plt.show()

```

Model Comparison (V2 – Correct Pipeline):

|                   | R2    | RMSE (log) | MAE (log) | RMSE (\$) | MAE (\$)  |
|-------------------|-------|------------|-----------|-----------|-----------|
| XGBoost           | 0.782 | 0.252      | 0.186     | 51206.000 | 34895.000 |
| Random Forest     | 0.777 | 0.255      | 0.188     | 52307.000 | 35534.000 |
| Linear Regression | 0.686 | 0.303      | 0.233     | 61579.000 | 43582.000 |
| Ridge Regression  | 0.686 | 0.303      | 0.233     | 61579.000 | 43582.000 |
| Lasso Regression  | 0.686 | 0.303      | 0.233     | 61583.000 | 43584.000 |



## Chunk 7: Hyperparameter Tuning — RandomizedSearchCV

RandomizedSearchCV vs GridSearchCV:

|              | GridSearchCV                            | RandomizedSearchCV                   |
|--------------|-----------------------------------------|--------------------------------------|
| How it works | Tries EVERY combination                 | Randomly samples N combinations      |
| Iterations   | $n_1 * n_2 * n_3 * \dots$ (exponential) | Fixed N (you choose)                 |
| Speed        | Impractical beyond 3-4 params           | Fast even with 8 params              |
| Quality      | Exhaustive but slow                     | Finds ~90% of the gain in 5% of time |

**Cross-validation during tuning:** RandomizedSearchCV uses k-fold CV on training data only. Test set is NEVER touched. This is the critical difference from naive manual tuning (where you might check test set repeatedly and overfit to it).

**We tune Random Forest and XGBoost** — the two models with real non-linear gains. Linear models are already optimal (RidgeCV/LassoCV handle their tuning internally).

```
In [32]: from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform
import time

SCORING = 'neg_root_mean_squared_error'
CV_FOLDS = 5
N_ITER = 50

print(f'Each model: {N_ITER} random combinations x {CV_FOLDS} CV folds')
```

Each model: 50 random combinations x 5 CV folds = 250 total fits

## Tuning — Random Forest

### Key hyperparameters:

- `n_estimators` : more trees = more stable predictions (diminishing returns above 300-500)
- `max_depth` : None = unlimited (can overfit). Shallow trees generalize better but underfit.
- `min_samples_leaf` : forces each leaf to represent at least N training rows. Prevents memorizing noise.
- `max_features` : fraction of features considered at each split. Lower = more diverse trees = better ensemble.

```
In [33]: rf_grid = {
    'n_estimators': randint(100, 501),
    'max_depth': [10, 20, 30, None],
    'min_samples_split': randint(2, 21),
    'min_samples_leaf': randint(1, 11),
    'max_features': uniform(0.3, 0.7),
}

rf_search = RandomizedSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    rf_grid, n_iter=N_ITER, cv=CV_FOLDS,
    scoring=SCORING, random_state=42, n_jobs=-1, verbose=1
)

t0 = time.time()
rf_search.fit(X_train_proc, y_train_log)
```

```
print(f'\nCompleted in {time.time()-t0:.1f}s')
print('Best params:')
for k, v in rf_search.best_params_.items():
    print(f' {k:22s}: {v}')
print(f'Best CV RMSE (log): {-rf_search.best_score_:.4f}')
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

Completed in 135.9s

Best params:

```
max_depth      : 30
max_features    : 0.42135525745544045
min_samples_leaf : 1
min_samples_split : 5
n_estimators    : 149
```

Best CV RMSE (log): 0.2545

```
In [34]: rf_tuned, rf_tuned_log, rf_tuned_dollar = evaluate(
        'Random Forest (Tuned)', rf_search.best_estimator_,
        X_train_proc, y_train_log, X_test_proc, y_test_log
    )
print(f'\nImprovement:')
print(f' R2      : {results_v2["Random Forest"]["R2"]:.4f} -> {results_v2["Random Forest (Tuned)"]["R2"]:.4f}')
print(f' RMSE    : ${results_v2["Random Forest"]["RMSE ($)"]:.0f} -> ${results_v2["Random Forest (Tuned)"]["RMSE ($)"]:.0f}')
```

```
=====
Random Forest (Tuned)
=====
R2      : 0.7807 (78.1% variance explained)
RMSE $  : $52,116
MAE $   : $35,392
```

Improvement:

```
R2      : 0.7770 -> 0.7807
RMSE $  : $52,307 -> $52,116
```

## Tuning — XGBoost

### Key hyperparameters:

- **learning\_rate** : contribution of each tree. Lower = more conservative, needs more trees. Rule: lower learning\_rate + more n\_estimators = better generalization.
- **max\_depth** : XGBoost trees should be shallow (3-6). Boosting + deep trees = overfitting.
- **subsample + colsample\_bytree** : row/feature sampling adds randomness similar to Random Forest. Reduces correlation between trees in the sequence.
- **reg\_alpha, reg\_lambda** : L1 and L2 penalty on leaf weights. Same concept as Lasso/Ridge but applied inside XGBoost's tree weight calculation.

- `min_child_weight`: minimum sum of instance weights in a leaf. Higher = more conservative splits.

```
In [35]: xgb_grid = {
    'n_estimators':    randint(200, 1001),
    'learning_rate':   uniform(0.01, 0.29),
    'max_depth':       randint(3, 9),
    'subsample':       uniform(0.6, 0.4),
    'colsample_bytree': uniform(0.5, 0.5),
    'reg_alpha':       uniform(0, 1),
    'reg_lambda':      uniform(0.5, 2.5),
    'min_child_weight': randint(1, 11),
}

xgb_search = RandomizedSearchCV(
    xgb.XGBRegressor(random_state=42, n_jobs=-1, verbosity=0),
    xgb_grid, n_iter=N_ITER, cv=CV_FOLDS,
    scoring=SCORING, random_state=42, n_jobs=-1, verbose=1
)
t0 = time.time()
xgb_search.fit(X_train_proc, y_train_log)
print(f'\nCompleted in {time.time()-t0:.1f}s')
print('Best params:')
for k, v in xgb_search.best_params_.items():
    lbl = f'{v:.4f}' if isinstance(v, float) else str(v)
    print(f'  {k:22s}: {lbl}')
print(f'Best CV RMSE (log): {-xgb_search.best_score_:.4f}')
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

/opt/anaconda3/lib/python3.12/site-packages/joblib/externals/loky/process\_executor.py:752: UserWarning: A worker stopped while some jobs were given to the executor. This can be caused by a too short worker timeout or by a memory leak.

warnings.warn(

Completed in 9.4s

Best params:

```
colsample_bytree    : 0.6999
learning_rate       : 0.0235
max_depth           : 6
min_child_weight    : 3
n_estimators        : 766
reg_alpha           : 0.9832
reg_lambda          : 1.6669
subsample           : 0.9440
```

Best CV RMSE (log): 0.2510

```
In [36]: xgb_tuned, xgb_tuned_log, xgb_tuned_dollar = evaluate(
    'XGBoost (Tuned)', xgb_search.best_estimator_,
    X_train_proc, y_train_log, X_test_proc, y_test_log
)
print(f'\nImprovement:')
print(f'  R2    : {results_v2["XGBoost"]["R2"]:.4f}  ->  {results_v2["X
```

```
print(f' RMSE : ${results_v2["XGBoost"]["RMSE ($)"]:, .0f} -> ${resu
```

```
=====
XGBoost (Tuned)
=====
```

```
R2      : 0.7857 (78.6% variance explained)
RMSE $  : $51,028
MAE $   : $34,710
```

Improvement:

```
R2      : 0.7823 -> 0.7857
RMSE $  : $51,206 -> $51,028
```

## Chunk 8: V1 vs V2 — Impact of Data Leakage

**This is the honest check.** We compare V1 (leaked) results with V2 (correct) results.

**If V1 R2 >> V2 R2:** leakage was significant — V1 results were artificially inflated **If**

**V1 R2 ≈ V2 R2:** leakage was mild for this specific dataset

**V1 results (from the leaked notebook):**

| Model             | V1 R2  | V1 RMSE (\$) |
|-------------------|--------|--------------|
| Linear Regression | 0.6856 | \$61,568     |
| Ridge Regression  | 0.6856 | \$61,568     |
| Lasso Regression  | 0.6856 | \$61,573     |
| Random Forest     | 0.7772 | \$52,272     |
| XGBoost           | 0.7829 | \$51,251     |

```
In [37]: v1_results = {
    'Linear Regression': {'R2': 0.6856, 'RMSE ($)': 61568},
    'Ridge Regression': {'R2': 0.6856, 'RMSE ($)': 61568},
    'Lasso Regression': {'R2': 0.6856, 'RMSE ($)': 61573},
    'Random Forest': {'R2': 0.7772, 'RMSE ($)': 52272},
    'XGBoost': {'R2': 0.7829, 'RMSE ($)': 51251},
}

print('='*65)
print('V1 (Leaked) vs V2 (Correct Pipeline) – Same Models')
print('='*65)
print(f'{"Model":<22} {"V1 R2":>8} {"V2 R2":>8} {"Diff":>8} {"V1 RMSE
```

```

for name in model_names:
    v1r2 = v1_results[name]['R2']
    v2r2 = results_v2[name]['R2']
    diff = v2r2 - v1r2
    v1rmse = v1_results[name]['RMSE ($)']
    v2rmse = results_v2[name]['RMSE ($)']
    sign = '+' if diff >= 0 else '-'
    print(f'{name:<22} {v1r2:>8.4f} {v2r2:>8.4f} {sign}{diff:>7.4f} $')

print('\nConclusion:')
print('The leakage in V1 had minimal impact on this dataset because:')
print(' - Only 207 rows (1%) had missing values -> imputation leakage')
print(' - Winsorization bounds barely shifted between full data and train-only')
print(' - However the PRINCIPLE matters: in other datasets leakage can be severe')

```

=====

V1 (Leaked) vs V2 (Correct Pipeline) – Same Models

=====

| Model             | V1 R2  | V2 R2  | Diff     | V1 RMSE   | V2 RMSE   |
|-------------------|--------|--------|----------|-----------|-----------|
| Linear Regression | 0.6856 | 0.6856 | + 0.0000 | \$ 61,568 | \$ 61,579 |
| Ridge Regression  | 0.6856 | 0.6856 | + 0.0000 | \$ 61,568 | \$ 61,579 |
| Lasso Regression  | 0.6856 | 0.6856 | + 0.0000 | \$ 61,573 | \$ 61,583 |
| Random Forest     | 0.7772 | 0.7770 | -0.0002  | \$ 52,272 | \$ 52,307 |
| XGBoost           | 0.7829 | 0.7823 | -0.0006  | \$ 51,251 | \$ 51,206 |

Conclusion:

The leakage in V1 had minimal impact on this dataset because:

- Only 207 rows (1%) had missing values -> imputation leakage was negligible
- Winsorization bounds barely shifted between full data and train-only
- However the PRINCIPLE matters: in other datasets leakage can be severe

## Save Best Model with Full Pipeline

**The correct way to save a model:** include the preprocessor in the saved artifact.

When you load this pipeline and call `predict(X_new)`, it will:

1. Impute missing values using train group medians
2. Winsorize using train IQR bounds
3. Log transform
4. Add distance features
5. Add ratio features



6. Drop columns
7. One-hot encode using train categories
8. Predict with the trained model

All in one call. This is production-ready.

```
In [38]: import joblib
from sklearn.pipeline import Pipeline as SKPipeline

# Build full pipeline: preprocessor + best model
best_model_name = max(results_v2, key=lambda k: results_v2[k]['R2'])
print(f'Best model: {best_model_name} (R2={results_v2[best_model_name]

model_map = {
    'Linear Regression': lr,
    'Ridge Regression': ridge,
    'Lasso Regression': lasso,
    'Random Forest': rf,
    'Random Forest (Tuned)': rf_search.best_estimator_,
    'XGBoost': xgb_m,
    'XGBoost (Tuned)': xgb_search.best_estimator_,
}

full_pipeline = SKPipeline([
    ('preprocessor', preprocessor),
    ('model', model_map[best_model_name])
])

joblib.dump(full_pipeline, 'best_model_v2_full_pipeline.pkl')
print('Saved to best_model_v2_full_pipeline.pkl')
print('\nUsage on new raw data:')
print(" pipeline = joblib.load('best_model_v2_full_pipeline.pkl')")
print(' pred_log = pipeline.predict(X_new_raw)')
print(' pred_dollar = np.expm1(pred_log)')
```

Best model: XGBoost (Tuned) (R2=0.7857)

Saved to best\_model\_v2\_full\_pipeline.pkl

Usage on new raw data:

```
pipeline = joblib.load('best_model_v2_full_pipeline.pkl')
pred_log = pipeline.predict(X_new_raw)
pred_dollar = np.expm1(pred_log)
```

---

## Project Summary — California Housing Prices V2 (Correct Pipeline)

---

# The Core Lesson of This Notebook

## Split first. Fit on train. Transform both.

Any transformation that learns a statistic from the data (median, IQR bounds, category list, mean, std) must be fitted **ONLY** on training data and then applied identically to the test set. Violating this causes data leakage — test performance looks better than real-world performance.

---

## Chunk 1 — Data Loading & Basic Quality Fixes

### What we did:

- Loaded raw CSV: 20,640 rows x 10 columns (9 numerical + 1 categorical)
- Identified 207 missing values in `total_bedrooms` (1.0% of rows)
- Computed statistical summary: skewness confirmed right-skew in count columns
- Removed 965 rows where `median_house_value >= $500,001` (US Census cap artifact)
- Converted `ocean_proximity` to category dtype

**Why census cap removal is safe before split:** This uses domain knowledge (fixed threshold = \$500,001, a known data collection artifact), NOT a statistic computed from the data. It is a data quality fix, not a learned transformation.

**Final shape after quality fix:** 19,675 rows x 10 columns

---

## Chunk 2 — Split First

**Train/test split: 80% train (15,740 rows) / 20% test (3,935 rows)**

Split happens **IMMEDIATELY** after basic quality fixes. All subsequent steps use training data only.

### Target transformation:

- `y_train_log = np.log1p(y_train)` — log transform the target
- `y_test_log = np.log1p(y_test)`
- This is safe before split because `log1p` is deterministic (no statistics learned)
- To reverse predictions: `np.expm1(prediction)`

**What leaked in V1 and why it matters in principle:**

| Transformation     | V1 (wrong)                         | V2 (correct)                              |
|--------------------|------------------------------------|-------------------------------------------|
| Imputation         | Group medians from all 19,675 rows | Group medians from 15,740 train rows only |
| Winsorization      | IQR bounds from all 19,675 rows    | IQR bounds from 15,740 train rows only    |
| One-hot categories | Determined from all 19,675 rows    | Determined from 15,740 train rows only    |

For this dataset the impact was negligible ( $R^2$  difference  $< 0.003$ ). In other datasets (small  $n$ , many missing values, high data skew) leakage can inflate  $R^2$  by 5-15%.

## Chunk 3 — EDA on Training Data Only

### Step 1: Target Distribution

- `log1p(median_house_value)` is approximately normally distributed after transformation
- QQ plot confirms slight left-tail deviation but overall near-normal
- D'Agostino K2 test:  $p < 0.05$  (non-normal) but on 15K rows this is expected even for near-normal data
- **Lesson:** Trust QQ plot over p-value on large datasets

### Step 2: Feature Distributions

- Count columns (total\_rooms, total\_bedrooms, population, households): severe right skew (skew  $> 3$ )
- `median_income`: moderate right skew (skew  $\sim 1.6$ )
- `housing_median_age`, `latitude`, `longitude`: approximately symmetric
- Conclusion: count columns and income need log transform

### Step 3: Correlation Heatmap

- Strongest correlation with target: `median_income` ( $r = 0.70$ )
- High inter-feature correlations: total\_rooms & total\_bedrooms ( $r = 0.94$ ), households & population ( $r = 0.91$ )
- lat/lon individually: weak correlation with target (0.06 to 0.17)

## Step 4: Scatter Plots

- `median_income` : clear positive diagonal band — strong linear relationship
- `latitude` , `longitude` : scattered, non-linear patterns individually
- Motivation: combine lat/lon into distance feature

## Step 5: Geographical Analysis

- Two main price clusters visible: San Francisco Bay Area and Los Angeles
- Confirms that proximity to major hubs drives price — motivates distance feature engineering

## Step 6: ANOVA + Effect Size (Eta Squared)

- F-statistic = 2337,  $p \approx 0$  — `ocean_proximity` means differ significantly
- **eta squared = 0.322** — `ocean_proximity` explains 32.2% of target variance (Large effect)
- Effect size interpretation:  $< 0.01$  trivial, 0.01-0.06 small, 0.06-0.14 medium,  $> 0.14$  large

## Step 7: Spearman Correlation with Effect Size

- `median_income` :  $\rho = 0.65$  (Large effect)
- Count columns (rooms, bedrooms, households, population): Small effect individually
- lat/lon individually: Trivial-Small effect
- All significant ( $p < 0.05$ ) — expected at this sample size

## Step 8: VIF Analysis

- Severe multicollinearity: `total_rooms` (VIF=84), `total_bedrooms` (VIF=47), `households` (VIF=71)
- Individual distance features (`dist_to_sf`, `dist_to_la`, `dist_to_sd`) would be severe if included
- **VIF  $\neq$  bad predictor.** High VIF means unstable coefficients in LINEAR models only. Trees unaffected.
- Plan: replace count columns with ratios, replace lat/lon with `dist_to_nearest_hub`

---

## Chunk 4 — Custom Sklearn Transformers (7

## Transformers)

All transformers follow the `BaseEstimator + TransformerMixin` pattern:

- `fit(X_train)` : learns statistics from training data
- `transform(X)` : applies using only the fitted statistics

| Transformer           | fit() learns                                  | transform() does                                  |
|-----------------------|-----------------------------------------------|---------------------------------------------------|
| GroupedMedianImputer  | Group medians by <code>ocean_proximity</code> | Fills NaN with the fitted group median            |
| Winsorizer            | IQR bounds per column                         | Clips values to fitted bounds                     |
| LogTransformer        | Nothing                                       | Applies <code>log1p</code> to specified columns   |
| DistanceFeatureAdder  | Nothing                                       | Computes Euclidean distance to SF/LA/SD           |
| RatioFeatureAdder     | Nothing                                       | Subtracts log columns (= ratio in original space) |
| ColumnDropper         | Nothing                                       | Drops specified columns                           |
| OceanProximityEncoder | Category list from training data              | Creates binary columns for each category          |

**Why OceanProximityEncoder needs fit():** If `pd.get_dummies()` is applied independently to train and test, a category present in test but not in train would create an extra column — shape mismatch and model failure. By fitting on train, we define exactly which columns to create and apply that same template to test.

## Chunk 5 — Preprocessing Pipeline

**Pipeline order:**

1. GroupedMedianImputer → fill 207 missing `total_bedrooms` values
2. Winsorizer → clip extreme values at train IQR bounds
3. LogTransformer → `log1p` on `count` + `income` columns
4. DistanceFeatureAdder → add `dist_to_sf`, `dist_to_la`, `dist_to_sd`, `dist_to_nearest_hub`
5. RatioFeatureAdder → add `rooms_per_household`, `bedrooms_per_room`, `population_per_household`
6. ColumnDropper → remove `lat`, `lon`, `individual`

distances, raw count columns  
 7. OceanProximityEncoder -> one-hot encode  
 ocean\_proximity (4 binary columns, drop reference)

**Result:** 10 clean, low-VIF features

**VIF after pipeline:** All features below 4.0 — multicollinearity fully resolved

**Final feature set:** housing\_median\_age, median\_income, dist\_to\_nearest\_hub, rooms\_per\_household, bedrooms\_per\_room, population\_per\_household, ocean\_proximity\_INLAND, ocean\_proximity\_ISLAND, ocean\_proximity\_NEAR BAY, ocean\_proximity\_NEAR OCEAN

---

## Chunk 6 — ML Models Results (V2)

| Model             | R2     | RMSE (\$) | MAE (\$)  |
|-------------------|--------|-----------|-----------|
| Linear Regression | ~0.686 | ~\$61,500 | ~\$43,500 |
| Ridge Regression  | ~0.686 | ~\$61,500 | ~\$43,500 |
| Lasso Regression  | ~0.686 | ~\$61,500 | ~\$43,500 |
| Random Forest     | ~0.777 | ~\$52,000 | ~\$35,400 |
| XGBoost           | ~0.783 | ~\$51,000 | ~\$34,800 |

### Key findings:

- Linear/Ridge/Lasso all identical: best alpha (Ridge) = 0.01 (almost no regularization needed — VIF engineering did its job)
  - Lasso zeroed 0 features: all 10 features carry genuine independent signal
  - Tree models +9 R2 points over linear: confirms genuine non-linear patterns in data
  - XGBoost vs Random Forest: ~\$1,000 better RMSE (difference grows with hyperparameter tuning)
- 

## Chunk 7 — Hyperparameter Tuning Results

**Method:** RandomizedSearchCV, 50 iterations, 5-fold CV on training data

**Why not GridSearchCV?** With 8 parameters each having 5-10 values =  $5^8 = 390,625$  combinations. RandomizedSearchCV finds 90% of the gain in 50

iterations.

**Random Forest tuning:** Searched `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`

**XGBoost tuning:** Searched `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`, `min_child_weight`

**Finding: Tuning gave minimal improvement (+0.2-0.4% R2)**

Why: The models are near their ceiling for this feature set. The remaining ~22% unexplained variance comes from information NOT in the dataset — neighborhood quality, school ratings, interior condition, exact street, renovation history. To push R2 above 0.85 you need richer features, not better tuning.

**Lesson: Feature engineering matters more than hyperparameter tuning.** The `dist_to_nearest_hub` feature alone lifted location correlation from 0.17 to 0.52. No tuning run can achieve that.

---

## Chunk 8 — V1 vs V2 Comparison

Results were virtually identical between the leaked V1 and the correct V2 pipeline.

**Why leakage was mild here:**

- Only 207 rows (1%) had missing values — imputation leakage was negligible
- IQR bounds barely shift when computed on 15,740 vs 19,675 rows

**Why you should always split first anyway:**

- In datasets with 20-30% missing values, imputation leakage is significant
  - In datasets with highly skewed distributions, Winsorization bounds shift noticeably
  - In time-series data, leakage from future rows into past rows is severe
  - In Kaggle competitions, even 0.002 R2 can mean the difference between top 10% and top 1%
  - It is a professional standard — every paper and production ML system does it this way
- 

## Key Lessons from This Project

1. **Split first — always.** Any transformation with a `fit()` step must be fitted on

train only. This is the single most important rule in the correct ML pipeline.

2. **sklearn Pipeline enforces correctness.** Once you build a Pipeline, sklearn handles the fit-on-train, transform-both protocol automatically.
3. **Custom transformers follow BaseEstimator + TransformerMixin.** fit() learns from data, transform() applies without re-learning.
4. **Grouped imputation beats global imputation.** Fill missing values within meaningful groups, not with a single global statistic. Same as Titanic age-by-title.
5. **Log transform changes distribution shape; scaling does not.** StandardScaler and MinMaxScaler rescale the axis — skewness remains. Log1p reshapes the distribution.
6. **VIF measures redundancy between features, not signal to target.** High VIF = unstable linear coefficients. Has no meaning for tree models.
7. **Effect size alongside p-value.** With large  $n$ , p-value is always significant. Eta squared and Spearman rho tell you the practical importance.
8. **Feature engineering beats hyperparameter tuning.** dist\_to\_nearest\_hub: correlation 0.17 -> 0.52. Ratio features: VIF 84 -> 2.6. No tuning achieves this.
9. **Model ceiling is set by the data.** R2 plateaus at 0.78-0.79 because ~22% of variance comes from information not in the dataset. More features, not better models, is the path forward.
10. **Save the full pipeline, not just the model.** joblib.dump(full\_pipeline) bundles preprocessing + model together. One call to predict() on raw data handles everything correctly.